23AID302 – BIG DATA ANALYTICS | 0TH REVIEW | TEAM B11

MediSpark

A Scalable Multi-Disease Clinical Risk Prediction Engine

Apache Spark · Hadoop HDFS · Scala · Spark MLlib · Docker

Suman Maitreya | CB.AI.U4AID24156 **Rajkumar R** | CB.AI.U4AID24143

Harish K | CB.AI.U4AID24118 **GRS Kartheek** | CB.AI.U4AID24116

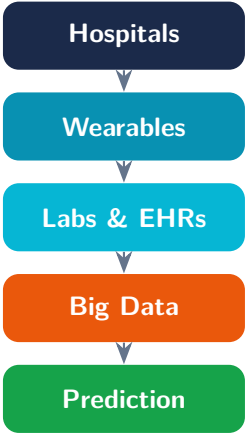
23AID302 – Big Data Analytics · Supervisor: Dr. Sreeja B P

Amrita Vishwa Vidyapeetham, Coimbatore

Introduction to the Problem Domain

Healthcare Meets Big Data

- The healthcare sector generates **millions of patient records daily** from hospitals, wearables, labs, and electronic health records (EHRs).
- Traditional systems **cannot process** this volume, velocity, and variety of data in real time.
- Chronic diseases — heart disease, diabetes, stroke, and lung cancer — cause **over 70% of global deaths** annually.
- Early detection through **data-driven prediction** can save millions of lives.
- Big Data technologies like **Apache Spark** and **Hadoop HDFS** now enable scalable, real-time healthcare analytics.



Importance of the Domain

Why Healthcare Big Data Matters

Volume

500+ million TB of health data generated daily worldwide

Velocity

ICU monitors generate real-time vitals every second

Variety

Multi-format data — labs, images, notes, wearables

Veracity

Messy, missing, inconsistent clinical data needs cleaning

Value: Predictive analytics can reduce misdiagnosis by 30% and enable early intervention for chronic diseases.

Source: WHO Global Health Observatory & IDC Data Age 2025 Report

Problem Statement

Existing healthcare prediction systems are designed for a **single disease**, operate on **static datasets**, and cannot process the massive volume, velocity, and variety of modern patient data **in real time** — leaving hospitals without a scalable, unified, **multi-disease early-warning system**.

✘ Gap 1

Single-disease models — no unified multi-disease prediction exists.

✘ Gap 2

Batch-only systems — cannot score new patient records in real time.

✘ Gap 3

No veracity handling — messy, missing, inconsistent clinical data ignored.

Literature Review — Related Work

Summary of Recent Research (2024–2025)

Paper 1: Rajendra Kumar et al. (2024) — *Biomedical Signal Processing & Control*, Elsevier

Method: Spark-based FCM clustering + Deep Belief Network feature fusion, classified via hybrid SqueezeNet-LinkNet model for heart disease prediction.

Gaps: Single-disease · Batch-only · No HDFS · Complex black-box deep learning model.

Paper 2: Fancy et al. (2025) — *Expert Systems*, Wiley

Method: Hadoop MapReduce with MRFO-based feature selection and RVM classifier tuned via Arithmetic Optimization Algorithm for healthcare big data.

Gaps: MapReduce (not Spark) · Single-disease · No streaming · No data-quality pipeline.

Paper 3: Chunduru et al. (2024) — *SN Computer Science*, Springer

Method: Flask web app with CNN + Random Forest as 7 isolated single-disease models (diabetes, heart, kidney, liver, cancer, malaria, pneumonia).

Gaps: No distributed computing · Isolated models · No real-time streaming · Small-scale data.

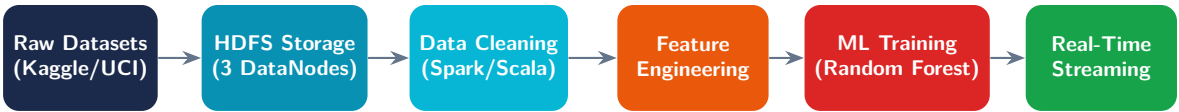
Research Gap

What Existing Systems Lack vs. What MediSpark Provides

What Exists	What MediSpark Provides
✘ Single-disease prediction models only	✔ 4 diseases predicted simultaneously
✘ No distributed HDFS-based storage	✔ Full HDFS storage with 3× replication
✘ No dedicated data veracity pipeline	✔ IQR outlier removal, null handling, dedup
✘ Streaming OR batch — never both	✔ Batch training + real-time Structured Streaming
✘ Complex infra (Kafka, Cassandra)	✔ Simple Docker Compose — fully reproducible
✘ No Scala-based ML Pipeline API	✔ Scala Spark ML Pipeline — production-grade

Proposed System — MediSpark Architecture

End-to-End Distributed Clinical Risk Prediction Engine



✔ How It Improves

- Processes **4 diseases** — Heart, Diabetes, Stroke, Lung Cancer — in one unified pipeline
- Scores new patients **every 5 seconds** via Spark Structured Streaming
- HDFS with **3× replication** for fault tolerance

★ What is New (Novelty)

- **Multi-disease parallelism** in a single Spark pipeline
- **Veracity-first** data quality stage with auditable reports
- **Native Scala** on JVM — zero Python serialization overhead
- **Docker-based** reproducible cluster setup

Objectives of the Project

O1 **To design** a distributed data storage and ingestion pipeline using Hadoop HDFS with 3× block replication across a Docker-based cluster.

O2 **To implement** a data quality and veracity pipeline in Scala for cleaning, validating, and reporting on multi-format healthcare data.

O3 **To engineer** feature transformation pipelines using Spark ML Pipeline API (StringIndexer, VectorAssembler, StandardScaler).

O4 **To train and evaluate** multi-disease risk prediction models using Spark MLlib Random Forest Classifier across 4 diseases.

O5 **To implement** real-time patient risk scoring using Spark Structured Streaming with a 5-second trigger interval.

O6 **To evaluate** model performance using Accuracy, Precision, Recall, F1-Score, and AUC-ROC metrics with results stored in HDFS.

Hardware / Software Requirements

Big Data Tools, Technologies & Datasets

Tools & Technologies

Tool	Purpose
Apache Spark 3.5.1	Distributed processing & ML
Hadoop HDFS 3.2.1	Distributed fault-tolerant storage
Scala 2.12.18	Primary pipeline language
Spark MLlib	Model training (Random Forest)
Spark ML Pipeline	Modular feature + model pipelines
Spark Streaming	Real-time patient risk scoring
Docker Compose	Reproducible multi-node cluster
sbt 1.9.7	Scala build tool & assembly
JDK 17 (Temurin)	Java runtime for Spark/Scala
Python 3.x	Visualization scripts

Datasets

Dataset	Rows	Source
Heart Disease	3.6M	UCI / Kaggle
Diabetes	3.7M	Kaggle
Stroke	4.4M	Kaggle
Lung Cancer	5.0M	Kaggle
Total	**16.8M**	

Original benchmark datasets augmented via bootstrapped resampling with Gaussian noise to achieve Big Data scale.

Raw Data Volume

1.2 GB raw → **3.68 GB** with 3× HDFS replication

Project Timeline

Milestone-Based Delivery Across Reviews

#	Phase	Target Review	Weeks
1	Literature Review & Problem Finalization	0th Review	1–2
2	Requirement Analysis & Dataset Collection	1st Review	3–4
3	Environment Setup — Hadoop, Spark, Docker Cluster	1st Review	4–5
4	Data Ingestion & HDFS Storage Pipeline	1st Review	5–6
5	Data Cleaning & Feature Engineering (Spark)	1st Review	6–7
6	ML Model Training — Random Forest	2nd Review	8–9
7	Real-Time Streaming Pipeline Integration	2nd Review	9–10
8	Testing, Evaluation & Performance Tuning	2nd Review	10–11
9	Final Documentation & Project Submission	2nd Review	11–12

Thank You

Questions and Feedback

MediSpark – Team B11

23AID302 – Big Data Analytics

Amrita Vishwa Vidyapeetham, Coimbatore

github.com/Suman-Maitreya/MediSpark